



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Utilisation de VUnit pour la simulation de bancs d'essai VHDL

Marc-Antoine MANNINGHAM

Automne 2025

Qu'est-ce que VUnit ?

VUnit est une librairie python et VHDL qui permet de faciliter la création et l'exécution de bancs d'essai VHDL. Elle permet de créer des tests unitaires à partir de banc d'essai VHDL. Ce genre de librairie est très utile pour automatiser les tests de validation de composants VHDL et de faire de l'intégration continue pour détecter les régressions sur les TBs.

Pourquoi utiliser VUnit ?

VUnit permet de:

- Compiler de manière incrémentale les fichiers VHDL pour réduire le temps de simulation
- Détecter et d'exécuter les tests unitaires de manière parallèle automatiquement
- Générer des rapports de tests et les exporter pour le CI (Gitlab Runner, Jenkins, etc.)
- Inclure des librairies VHDL pour faciliter la création de tests unitaires
- Avoir des composantes de tests pour des protocoles (AXI, UART, etc.)
- Être indépendant d'un outil de simulation particulier

Installation de VUnit

L'installation consiste à installer la librairie python `vunit_hdl`. Une manière de l'installer est d'utiliser `pip`:

```
pip install vunit_hdl
```

Prérequis

Pour utiliser VUnit, il faut avoir installé un outil de simulation VHDL (ActiveHDL, GHDL, etc.). Pour avoir une interface graphique pour les tests avec GHDL, il faut avoir installé l'outil GTKWave.

```
sudo apt install gtkwave ghdl # Installation pour Ubuntu  
sudo dnf install gtkwave ghdl # Installation pour CentOS/Fedora
```

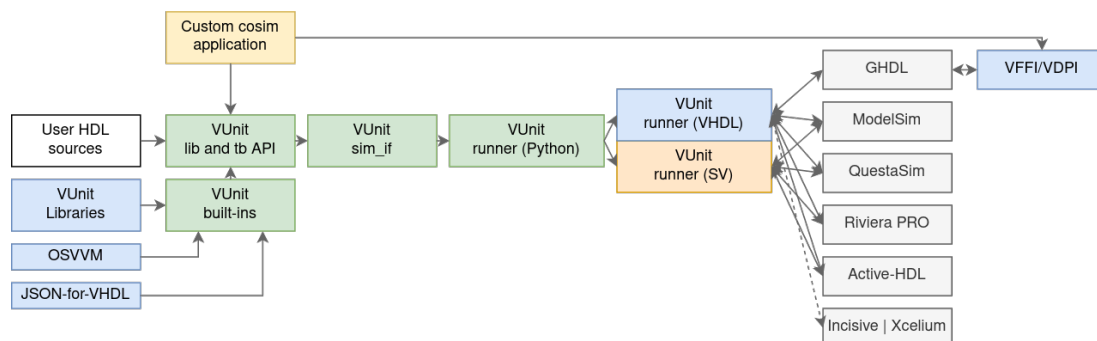


Fig. 1. – Diagramme de fonctionnement de VUnit

Démarrage rapide avec VUnit

Script python

Un projet avec VUnit consiste à créer un script python à la racine qui va définir où se trouvent les fichiers VHDL et comment les compiler. Voici un exemple de script python pour un projet simple:

```
# Fichier run.py
from vunit import VUnit

vu = VUnit.from_argv()
vu.add_vhdl_builtins() # Permet d'inclure les librairies VHDL de VUnit

lib = vu.add_library("lib")
lib.add_source_files("src/*.vhdl") # Ajouter les bons fichiers VHDL ici

vu.main()
```

Exemple d'entité en VHDL à tester (ne dépend pas de VUnit)

Voici un exemple d'entité en VHDL à tester avec VUnit. Cette entité est un simple inverseur de signal:

```
-- Fichier src/simple_alu.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity gate_not is
  port
  (
    i : in std_logic;
    o : out std_logic
  );
end gate_not;

architecture rtl of gate_not is
begin
  o <= not i;
end rtl;
```

Fichiers de banc d'essai VHDL

Un banc d'essai VHDL doit être écrit de manière à être compatible avec VUnit. Pour ce faire, il faut qu'il contienne un générique `runner_cfg` dans l'entité de test, une déclaration de librairie `vunit_lib` et des appels à la fonction `run` dans un `while` avec la `test_suite`.

```
-- Fichier src/simple_alu_tb.vhdl
library vunit_lib; -- Nécessaire pour VUnit
context vunit_lib.vunit_context; -- Nécessaire pour VUnit

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

library lib;

entity simple_alu_tb is
    generic
        (runner_cfg : string); -- Nécessaire pour vunit
end entity;

architecture tb of simple_alu_tb is
    signal i, o : std_logic;
begin
    alu_uut : entity lib.gate_not
        port map
        (
            i => i,
            o => o
        );

    main : process is
    begin
        test_runner_setup(runner, runner_cfg); -- Nécessaire pour vunit
        while test_suite loop -- Nécessaire pour vunit
            if run("inversion_0") then -- Exemple de test réussi
                i <= '0';
                wait for 10 ns;
                assert o = '1' report "Expected 1 but got " & to_string(o);
            elsif run("inversion_1") then -- Exemple de test échoué
                i <= '0'; -- Devrait être '1'
                wait for 10 ns;
                assert o = '0' report "Expected 0 but got " & to_string(o);
            end if;
        end loop;
        test_runner_cleanup(runner); -- Nécessaire pour vunit
    end process;
end architecture;
```

Commande pour utiliser VUnit

Pour lancer tous les tests sans interface:

```
python3 run.py
```

Pour lancer un test spécifique:

```
python3 run.py lib.simple_alu_tb.inversion_0
# À noter qu'une étoile (*) peut être utilisée pour lancer plusieurs tests qui
ont un nom commun
python3 run.py lib.simple_alu_tb.inversion*
```

Pour lancer un test avec une interface graphique:

```
python3 run.py -g lib.simple_alu_tb.inversion_0
```

Bonnes pratiques

Il est en général recommandé de faire un fichier de banc d'essai par entité VHDL à tester. Cela permet de mieux organiser les tests. N'oubliez pas que le VHDL contient des procédures et des fonctions qui permettent d'éviter de se répéter.

Lien vers la documentation de VUnit (en anglais)

Pour plus de références, la documentation de VUnit est disponible sur le site officiel: <https://vunit.github.io/>

Pour aller plus loin

- Utilisation de VUnit avec des génériques sur le banc d'essai: https://vunit.github.io/blog/2017_06_03_enable_your_simulator_to_handle_complex_top_level_generics.html
- Utilisation de VUnit avec des protocoles (AXI, UART, etc.): https://vunit.github.io/verification_components/user_guide.html/